



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71251230
Nama Lengkap	Okky Alexander
Minggu ke / Materi	12 / Tipe Data Dictionary

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI DICTIONARY

Dictionary memiliki kemiripan dengan list, namun bersifat lebih fleksibel. Perbedaan utamanya terletak pada jenis indeks yang digunakan list hanya menerima indeks berupa integer, sedangkan dictionary bisa menggunakan tipe data apa pun sebagai indeks. Setiap anggota dictionary terdiri dari pasangan kunci dan nilai, di mana setiap kunci harus bersifat unik. Tidak diperbolehkan ada dua kunci yang sama dalam satu dictionary.

Dictionary dapat dipahami sebagai pemetaan antara sekumpulan kunci dan sekumpulan nilai, di mana setiap kunci mengarah pada satu nilai tertentu. Hubungan antara keduanya disebut pasangan kunci-nilai (*key-value pair*) atau item. Sebagai contoh, dictionary bisa digunakan untuk memetakan kata-kata bahasa Inggris ke bahasa Spanyol, dengan asumsi setiap kata memiliki satu padanan.

Untuk membuat dictionary kosong, Python menyediakan fungsi bawaan `dict()`. Penggunaannya sebagai nama variabel sebaiknya dihindari agar tidak terjadi konflik. Tanda kurung kurawal `{}` merepresentasikan dictionary kosong, sedangkan untuk menambahkan item digunakan tanda kurung kotak `[]`:

```
>>> diti = dict()
```

```
>>> diti['one'] = 'siji'
```

```
>>> print(diti)
```

```
{'one': 'siji'}
```

Dictionary juga bisa langsung diisi saat pertama kali dibuat:

```
>>> diti = {'one': 'siji', 'two': 'loro', 'three': 'telu'}
```

```
>>> print(diti)
```

```
{'one': 'siji', 'two': 'loro', 'three': 'telu'}
```

Urutan item yang ditampilkan tidak selalu sesuai urutan input, namun hal ini tidak menjadi masalah karena akses dilakukan melalui kunci, bukan indeks angka. Apabila kunci yang dicari tidak ditemukan, Python akan memunculkan pesan kesalahan `KeyError`. Fungsi `len()` digunakan untuk mengetahui jumlah pasangan kunci-nilai dalam dictionary.

Operator `in` pada dictionary digunakan untuk mengecek apakah suatu kunci tersedia, dan hasilnya berupa `True` atau `False`. Untuk mengecek keberadaan suatu nilai, dapat digunakan method `values()` yang dikonversi terlebih dahulu ke dalam list.

Perbedaan penting lainnya adalah cara kerja operator `in` pada list dan dictionary. List menggunakan pencarian linear yang semakin lambat seiring bertambahnya data, sedangkan dictionary menggunakan algoritma Hash Table yang waktu pencariannya tetap konstan meskipun jumlah item terus bertambah. Inilah salah satu keunggulan utama dictionary dibandingkan list dalam hal efisiensi pencarian data.

MATERI DICTIONARY SEBAGAI PENGHITUNG

Misalkan kita memiliki sebuah string dan perlu menghitung kemunculan setiap huruf di dalamnya. Ada tiga pendekatan yang bisa digunakan: pertama, membuat 26 variabel untuk masing-masing huruf alfabet; kedua, membuat list berisi 26 elemen lalu mengonversi setiap karakter menjadi angka sebagai indeks; ketiga, menggunakan dictionary dengan karakter sebagai kunci dan jumlah kemunculan sebagai nilainya.

Ketiga cara tersebut menghasilkan output yang sama, namun dengan proses yang berbeda. Pendekatan menggunakan dictionary dinilai paling praktis karena kita tidak perlu mengetahui terlebih dahulu huruf apa saja yang akan muncul dalam string cukup sediakan ruang untuk huruf yang benar-benar muncul. Berikut contoh implementasinya:

```
word = 'dinoajur'
d = dict()
for c in word:
    if c not in d:
        d[c] = 1
    else:
        d[c] = d[c] + 1
print(d)
```

Gambar 12.1: Code Program histogram

Pola komputasi seperti ini disebut histogram, sebuah istilah dalam statistika yang merujuk pada kumpulan frekuensi kemunculan. Cara kerjanya: perulangan `for` menelusuri setiap karakter dalam string. Bila

karakter tersebut belum ada di dictionary, maka item baru dibuat dengan nilai awal 1. Sebaliknya, jika karakter sudah tercatat, nilainya langsung bertambah satu. Hasil outputnya adalah:

```
{'d': 1, 'i': 1, 'n': 1, 'o': 1, 'a': 1, 'j': 1, 'u': 1, 'r': 1}
```

Dictionary juga memiliki method bernama `get()` yang menerima dua argumen: kunci dan nilai default. Jika kunci ditemukan, method ini mengembalikan nilai yang sesuai; jika tidak, nilai default yang dikembalikan. Contohnya:

```
>>> counts = {'chuck': 1, 'asui': 42, 'jan': 100}
```

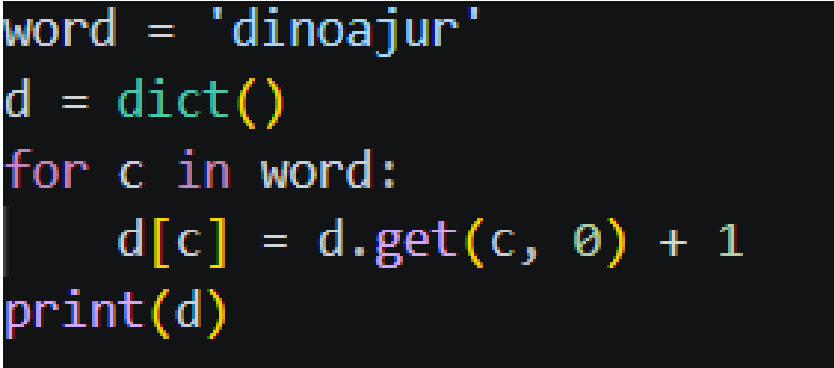
```
>>> print(counts.get('jan', 0))
```

```
100
```

```
>>> print(counts.get('tim', 0))
```

```
0
```

Dengan memanfaatkan `get()`, kode histogram di atas bisa dipersingkat secara signifikan karena method ini otomatis menangani kondisi saat kunci belum ada dalam dictionary:



```
word = 'dinoajur'
d = dict()
for c in word:
    d[c] = d.get(c, 0) + 1
print(d)
```

Gambar 12.2: Code Program

Empat baris kondisi if berhasil dipadatkan menjadi satu baris saja. Hasil outputnya tetap sama. Penggunaan `get()` untuk menyederhanakan loop penghitungan seperti ini sudah menjadi idiom yang sangat umum dalam pemrograman Python.

MATERI DICTIONARY DAN FILE

Salah satu kegunaan umum dictionary adalah menghitung frekuensi kemunculan kata dalam file teks. Programnya bekerja dengan membaca file baris per baris menggunakan *nested loop* perulangan luar membaca tiap baris, perulangan dalam menelusuri setiap kata di baris tersebut lalu mencatatnya ke dalam dictionary.

```
fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

print(counts)
```

Gambar 12.3: Code Program

Penulisan `counts[word] += 1` merupakan bentuk ringkas dari `counts[word] = counts[word] + 1`. Python juga mendukung operator sejenis seperti `-=`, `*=`, dan `/=`. Output yang dihasilkan berupa seluruh data frekuensi kata dalam urutan tidak tersortir, misalnya kata 'and' muncul 3 kali dan 'sun' 2 kali.

MATERI LOOPING DAN DICTIONARY

Saat digunakan dalam perulangan `for`, dictionary akan menelusuri setiap kunci yang ada di dalamnya secara otomatis. Setiap kunci beserta nilainya dapat dicetak langsung menggunakan operator indeks:

```
counts = {'chuck': 1, 'annie': 42, 'jan': 100}
```

```
for key in counts:
```

```
    print(key, counts[key])
```

Perlu diingat bahwa urutan kunci yang ditampilkan tidak mengikuti pola tertentu. Perulangan ini juga bisa dikombinasikan dengan kondisi tertentu, misalnya hanya menampilkan data dengan nilai di atas 10:

```
for key in counts:
```

```
    if counts[key] > 10:
```

```
        print(key, counts[key])
```

Apabila ingin menampilkan kunci secara berurutan alfabetis, langkahnya adalah mengubah kunci dictionary menjadi list menggunakan method `keys()`, mengurutkannya dengan `sort()`, lalu melakukan perulangan pada list yang sudah terurut tersebut:

```
lst = list(counts.keys())
lst.sort()
for key in lst:
    print(key, counts[key])
```

Gambar 12.4: Code Program

Hasilnya, pasangan kunci-nilai akan ditampilkan dalam susunan alfabetis yang rapi.

MATERI ADVANCED TEXT PARSING

Pada bagian sebelumnya teks yang digunakan sudah bersih dari tanda baca. Kini kita akan mencoba memproses file teks lengkap dengan tanda baca. Masalah yang muncul adalah fungsi `split()` bawaan Python memisahkan kata berdasarkan spasi, sehingga "soft!" dan "soft" dianggap dua kata berbeda. Hal serupa berlaku pada huruf kapital "Who" dan "who" dihitung secara terpisah.

Untuk mengatasi hal ini, Python menyediakan beberapa method string seperti `lower()`, dan yang paling berguna adalah `translate()`. Method ini bekerja berdasarkan dokumentasi berikut:

```
line.translate(str.maketrans(fromstr, tostr, deletestr))
```

Fungsinya mengganti karakter pada `fromstr` dengan karakter di posisi yang sama pada `tostr`, sekaligus menghapus semua karakter yang terdapat dalam `deletestr`. Dalam kasus ini, kita memanfaatkan parameter `deletestr` untuk menghapus seluruh tanda baca. Daftar tanda baca yang dikenali Python dapat dilihat melalui:

```
>>> import string
```

```
>>> string.punctuation
```

```
'!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

Implementasi lengkapnya adalah sebagai berikut:

```
import string

fname = input('Enter the file name: ')
try:
    fhand = open(fname)
except:
    print('File cannot be opened:', fname)
    exit()

counts = dict()
for line in fhand:
    line = line.rstrip()
    line = line.translate(line.maketrans('', '', string.punctuation))
    line = line.lower()
    words = line.split()
    for word in words:
        if word not in counts:
            counts[word] = 1
        else:
            counts[word] += 1

print(counts)
```

Gambar 12.5: Code Program

Setiap baris diproses dengan menghapus tanda baca, mengubah ke huruf kecil, lalu memecahnya menjadi kata-kata sebelum dihitung frekuensinya. Hasilnya, kata-kata seperti 'romeo' tercatat muncul 40 kali dan 'juliet' sebanyak 32 kali dalam file romeo-full.txt.

Source Github : https://github.com/tuangkeman/71251230_Okky.git

LATIHAN 12.1

```
dictionary = {1:10,2:20,3:30,4:40,5:50,6:60}
urutan = 0
print("key ", "value ", "item ")
for key , value in dictionary.items():
    urutan += 1
    print(f"{key}      {value}      {urutan}")
```

Gambar 12.6: Code Program

Kode ini dimulai dengan membuat sebuah dictionary bernama dictionary yang berisi 6 pasang key-value, yaitu 1:10, 2:20, 3:30, 4:40, 5:50, 6:60 di mana setiap key adalah bilangan bulat dan valuenya merupakan kelipatan 10 dari key tersebut.

Selanjutnya, variabel urutan diinisialisasi dengan nilai 0, yang nantinya akan digunakan sebagai penghitung (counter) untuk melacak urutan iterasi.

Sebelum perulangan dimulai, program mencetak header kolom "key", "value", dan "item" menggunakan print() biasa sebagai judul dari output yang akan ditampilkan.

Kemudian, program melakukan perulangan menggunakan for key, value in dictionary.items(), yang berarti setiap iterasi akan mengambil pasangan key dan value dari dictionary secara bersamaan. Di dalam perulangan, nilai urutan ditambah 1 setiap kali iterasi berjalan (urutan += 1), lalu mencetak key, value, dan nomor urutan saat itu menggunakan f-string dengan format {key}, {value}, dan {urutan}.

Hasil akhirnya adalah tabel yang menampilkan setiap key dan value dari dictionary beserta nomor urutannya dari 1 hingga 6.

LATIHAN 12.2

```
lista = ['red', 'green', 'blue']
listb = [■ '#FF0000', ■ '#008000', ■ '#0000FF']
order = [1,2,0]
ba = [lista[i] for i in order]
bb = [listb[i] for i in order]
dik = dict(zip(ba, bb))
print(dik)
```

Gambar 12.7: Code Program

Kode ini diawali dengan mendefinisikan dua buah list. lista berisi nama-nama warna dalam bentuk teks yaitu 'red', 'green', dan 'blue', sedangkan listb berisi kode warna dalam format heksadesimal yaitu

'#FF0000', '#008000', dan '#0000FF'. Kedua list ini saling berkorespondensi, artinya 'red' berpasangan dengan '#FF0000', 'green' dengan '#008000', dan seterusnya.

Kemudian dibuat list `order = [1, 2, 0]` yang berfungsi sebagai daftar indeks dengan urutan yang telah diubah. Urutan ini tidak dimulai dari 0, melainkan dimulai dari indeks 1, lalu 2, lalu 0 sehingga nantinya urutan elemen yang diambil akan berubah.

Selanjutnya, `ba` dan `bb` dibuat menggunakan list comprehension. `ba` mengambil elemen dari `lista` berdasarkan urutan indeks di `order`, sehingga hasilnya adalah `['green', 'blue', 'red']`. Begitu pula `bb` mengambil elemen dari `listb` dengan cara yang sama, menghasilkan `['#008000', '#0000FF', '#FF0000']`.

Terakhir, dik dibuat dengan menggabungkan `ba` dan `bb` menggunakan fungsi `zip()` yang memasangkan elemen-elemen dari kedua list secara beurut, lalu dikonversi menjadi dictionary menggunakan `dict()`. Hasil akhir yang dicetak adalah `{'green': '#008000', 'blue': '#0000FF', 'red': '#FF0000'}` sebuah dictionary yang memetakan nama warna ke kode heksadesimalnya, namun dengan urutan yang telah digeser sesuai `order`.

LATIHAN 12.3

```
nama_file = input("Masukkan nama file : ")
#Minggu 12\mbox-short.txt
counts = {}

with open(nama_file) as f:
    lines = f.readlines()

for line in lines:
    if line.startswith("From "):
        parts = line.split()
        email = parts[1]
        counts[email] = counts.get(email, 0) + 1

print(counts)
```

Gambar 12.8: Code Program

Kode ini dimulai dengan meminta pengguna memasukkan nama file melalui `input()`, lalu disimpan ke variabel `nama_file`. Komentar di bawahnya (`#Minggu 12\mbox-short.txt`) menunjukkan contoh nama file yang dimaksud. Selanjutnya, dibuat sebuah dictionary kosong bernama `counts` yang akan digunakan untuk menyimpan hasil perhitungan.

Kemudian, program membuka file tersebut menggunakan `with open(nama_file) as f`, yang merupakan cara aman membuka file karena file akan otomatis ditutup setelah blok selesai dijalankan. Semua baris dalam file dibaca sekaligus menggunakan `f.readlines()` dan disimpan ke dalam list `lines`.

Setelah itu, program melakukan perulangan untuk setiap baris dalam lines. Di dalam perulangan, terdapat kondisi if `line.startswith("From ")` yang hanya memproses baris-baris yang diawali dengan kata "From " ini adalah format umum pada file mbox (kotak surat email) yang menandai asal pengirim. Jika kondisi terpenuhi, baris tersebut dipecah menjadi beberapa bagian menggunakan `line.split()`, lalu elemen ke-1 (indeks 1) diambil sebagai alamat email pengirim dan disimpan ke variabel email.

Baris `counts[email] = counts.get(email, 0) + 1` adalah inti dari logika penghitungan. Fungsi `counts.get(email, 0)` akan mencari apakah email tersebut sudah ada di dictionary jika sudah ada, ambil nilainya; jika belum, gunakan nilai default 0. Lalu nilai tersebut ditambah 1, sehingga setiap kali email yang sama muncul, hitungannya akan bertambah.

Terakhir, `print(counts)` mencetak seluruh isi dictionary yang berisi daftar alamat email beserta jumlah kemunculannya di dalam file tersebut.

LATIHAN 12.4

```
nama_file = input("Masukkan nama file : ")
#Minggu 12\mbox-short.txt
counts = {}

try:
    with open(nama_file) as f:
        lines = f.readlines()

        for line in reversed(lines):
            if line.startswith("From "):
                parts = line.split()
                email = parts[1]
                domain = email.split("@")[1]
                counts[domain] = counts.get(domain, 0) + 1

except FileNotFoundError:
    print("File tidak ditemukan")
    exit()

print(counts)
```

Gambar 12.9: Code Program

Kode ini memiliki struktur yang mirip dengan sebelumnya, namun terdapat beberapa perbedaan dan penambahan yang penting.

Seperti sebelumnya, program meminta input nama file dari pengguna dan menyiapkan dictionary kosong counts untuk menampung hasil perhitungan.

Perbedaan pertama adalah penggunaan blok try dan except. Seluruh proses membaca file dan penghitungan dibungkus dalam blok try, sehingga jika terjadi kesalahan misalnya file tidak ditemukan program tidak akan langsung crash. Jika file memang tidak ada, blok except FileNotFoundError akan menangkap error tersebut, mencetak pesan "File tidak ditemukan", lalu menghentikan program dengan exit(). Ini membuat kode jauh lebih aman dan ramah pengguna.

Perbedaan kedua ada pada perulangan, yaitu penggunaan reversed(lines). Fungsi ini membalik urutan pembacaan baris, sehingga baris terakhir dalam file akan diproses lebih dulu. Meski demikian, logika penghitungannya tetap sama.

Perbedaan ketiga dan paling utama adalah pada bagian ekstraksi data. Setelah mendapatkan alamat email dari parts[1], kode ini tidak langsung menghitung email-nya, melainkan mengambil nama domain-nya saja menggunakan email.split("@")[1]. Misalnya dari email user@gmail.com, yang diambil hanya gmail.com. Domain tersebut kemudian disimpan ke variabel domain dan dihitung kemunculannya menggunakan counts.get(domain, 0) + 1.

Jadi, hasil akhir yang dicetak oleh print(counts) adalah dictionary yang berisi daftar domain email beserta jumlah kemunculannya, bukan alamat email lengkap seperti pada kode sebelumnya.